PIAZZA DEI MESTIERI – TORINO

IeFP – Operatore Informatico | Seconda Annualità (Classe B22-12-2026-0)

PROGRAMMAZIONE 2

Sviluppo Software Avanzato, OOP e Metodologie di Progetto

Manuale Didattico Ufficiale del Corso di Laboratorio

Docente: Daniele Cerrina
Email: daniele.cerrina@immaginazioneelavoro.it
Anno Formativo: 2026 / 2027

Programmazione 2 – Sviluppo Software e Metodologie
Percorso triennale IeFP per la Qualifica di Operatore Informatico.
© 2026–2027 Piazza dei Mestieri / Immaginazione e Lavoro.
Docente: Daniele Cerrina.

Documento didattico in continuo aggiornamento durante l'anno formativo.
Compilato con LaTeX e pdflatex.

Presentazione del Volume

Questo volume accompagna gli studenti della seconda annualità del corso per **Operatore Informatico** presso la Piazza dei Mestieri di Torino nel percorso avanzato di **Programmazione 2**.

Durante questo secondo anno il focus si sposta dall'apprendimento della sintassi di base all'ingegneria del software applicata:

- Progettazione a oggetti (Classi, incapsulamento, ereditarietà).
- Gestione di strutture dati avanzate e collezioni complesse.
- Manipolazione di file e persistenza dello stato.
- Gestione delle eccezioni, testing e debugging avanzato.
- Sviluppo cooperativo e architettura di un progetto software completo.

Come consultare questo documento: Il libro è strutturato con un **indice interattivo e cliccabile**. Cliccando su qualunque voce dell'indice o dei segnalibri PDF è possibile saltare istantaneamente alla sezione d'interesse.

Indice Generale

Presentazione del Volume ii

I Metodologia di Sviluppo e Consolidamento 1

1 Benvenuti al Secondo Anno: Metodologia e OOP 2
1.1 Dal Pensiero Algoritmico allo Sviluppo Software . . . 2
1.2 I Pilastri del Programma Annuale . . . 2
1.3 Standard di Laboratorio e Buone Pratiche . . . 3
1.4 Esercizio di Riscaldamento: Verso gli Oggetti . . . 3

2 I Grandi Concetti della Programmazione 4
2.1 Il Modello Computazionale e la Mappa dei Concetti . . . 4
2.1.1 Che cos'è formalmente un Algoritmo? . . . 5
2.1.2 Lo Schema Universale: Input → Algoritmo → Output . . . 5
2.1.3 Le Tre Regole Fondamentali di un Algoritmo . . . 5
2.2 La Gestione della Memoria: Variabili, Costanti e Tipi . . . 6
2.2.1 La Dinamica dell'Assegnazione: Da Destra a Sinistra . . . 6
2.2.2 Le Costanti: Valori Protetti in Sola Lettura . . . 7
2.2.3 Il Catalogo dei Tipi di Dato Fondamentali . . . 8
2.3 Aritmetica del Calcolatore e l'Operatore Modulo (%) . . . 9
2.3.1 L'Operatore Speciale Modulo (%) . . . 9
2.4 La Logica Booleana e il Controllo di Flusso . . . 10
2.4.1 Gli Operatori Relazionali di Confronto . . . 10
2.4.2 Il Tranello Mortale: Assegnazione (=) contro Confronto (==) . . . 10
2.4.3 Gli Operatori Logici Booleani: AND, OR, NOT . . . 11
2.4.4 Espressioni Booleane Composte e la Precedenza delle Parentesi . . . 12
2.4.5 Le Strutture di Selezione: IF, ELSE e Scelte Multiple . . . 12
2.5 L'Automazione: Cicli e Iterazioni . . . 13
2.5.1 Il Ciclo Pre-Condizionale (WHILE) . . . 13
2.5.2 Il Ciclo a Conteggio Prefissato (FOR) . . . 14
2.5.3 Guida di Scelta Architetturale: WHILE o FOR? . . . 14
2.6 Collezioni e Sequenze: Vettori (Array) e Stringhe . . . 14
2.6.1 La Regola Sacra dell'Indice da ZERO . . . 15
2.6.2 Scorrere un Vettore: Il Binomio Perfetto FOR + Array . . . 16
2.6.3 Le Stringhe come Vettori Ordinati di Caratteri . . . 16
2.7 Laboratorio Pratico: Le Tre Sfide alla LIM e Debugging . . . 16
2.7.1 Sfida 1: Il Tranello nel Controllo Condizionale . . . 17
2.7.2 Sfida 2: L'Iterazione Paralizzata nel Ciclo . . . 17
2.7.3 Sfida 3: L'Accesso Oltre il Confine del Vettore . . . 18
2.8 Modellazione Completa del Vending Machine Manager . . . 18

2.8.1 1. Modellazione della Struttura Dati di Memoria 19
2.8.2 2. L'Algoritmo Unificato di Acquisto ed Erogazione 19

Riferimento Rapido e Cheat-Sheet **21**

Part I

Metodologia di Sviluppo e Consolidamento

1

Benvenuti al Secondo Anno: Metodologia e OOP

> *"Scrivere codice che un computer comprende è facile. Scrivere codice che gli esseri umani sanno leggere, mantenere ed estendere nel tempo è la vera arte della programmazione."*

Corso: Programmazione 2 – Laboratorio di Sviluppo Software
Obiettivo del secondo anno: Passare dalla programmazione procedurale di base alla progettazione modulare e orientata agli oggetti, acquisendo standard professionali di sviluppo.

1.1 Dal Pensiero Algoritmico allo Sviluppo Software

Nel corso della prima annualità abbiamo esplorato i mattoni fondamentali dell'informatica: la sequenza logica, le condizioni (`if/else`), i cicli (`while`, `for`) e le prime funzioni.

Nel secondo anno compiamo un salto di qualità decisivo: non ci limiteremo a scrivere script che "funzionano", ma impareremo a progettare **architetture software robuste**, manutenibili e modulari.

L'Approccio del Secondo Anno

Un programma professionale non è un monolite disordinato di centinaia di righe, ma un sistema ordinato di componenti indipendenti che collaborano tra loro scambiandosi messaggi e dati validati.

1.2 I Pilastri del Programma Annuale

Il percorso di Programmazione 2 si articolerà attorno a quattro direttrici principali:

1. **Programmazione Orientata agli Oggetti (OOP):**
   - Concetto di Classe vs Istanza (Oggetto).
   - Incapsulamento: attributi, metodi e protezione dello stato interno.
   - Ereditarietà e Polimorfismo: riutilizzo intelligente del codice.
2. **Strutture Dati Dinamiche e Collezioni:**

- Liste avanzate, dizionari (tabelle chiave-valore), insiemi e tuple.
- Algoritmi di ricerca e ordinamento a confronto.

3. **Persistenza dei Dati e Gestione degli Errori:**
   - Lettura e scrittura di file (testo, CSV, JSON).
   - Gestione professionale delle eccezioni (`try / catch / finally`) per prevenire crash imprevisti.
4. **Progetto Pratico di Fine Anno:**
   - Realizzazione in team di un applicativo completo, con interfaccia utente, gestione dello stato e salvataggio dei dati.

1.3 Standard di Laboratorio e Buone Pratiche

Durante tutte le sessioni di laboratorio adotteremo rigorosi standard professionali:

Le Regole d'Oro del Laboratorio

1. **Nomi parlanti:** I nomi di variabili e funzioni devono spiegare *cosa fanno*, non essere sigle oscure come `a`, `x1`, `temp2`.
2. **Una funzione, un solo compito:** Ogni funzione deve avere una responsabilità chiara e circoscritta.
3. **Controllo di Versione (Git):** Ogni sessione di lavoro prevede il commit delle modifiche con messaggi chiari e significativi.

1.4 Esercizio di Riscaldamento: Verso gli Oggetti

Per riscaldare la logica prima di addentrarci nella OOP formale, analizziamo come rappresentare un'entità del mondo reale.

Modellare uno Studente

Immagina di dover gestire i dati di una classe di studenti.

1. Quali caratteristiche (dati) identificano univocamente ogni studente?
2. Quali azioni (operazioni) possono essere compiute su uno studente (es. aggiungere un voto, calcolare la media)?

Dalla Scheda al Modello Dati

Nel paradigma a oggetti separeremo nettamente:

- **Stato (Attributi):** `nome`, `cognome`, `matricola`, `elenco_voti`.
- **Comportamento (Metodi):** `aggiungi_voto(voto)`, `calcola_media()`, `is_promosso()`.

Nel prossimo capitolo tradurremo questo modello nella nostra prima classe software!

2

I Grandi Concetti della Programmazione

> *"I linguaggi di programmazione cambiano e le tecnologie si evolvono, ma i principi fondamentali della logica, della memoria e delle strutture di controllo restano invariati ovunque si scriva codice."*

Riferimento Didattico: Lezione del 22 Settembre 2026 – Laboratorio di Programmazione 2
Progetto Guida Annuale: Vending Machine Manager (Distributore Automatico Modulare)
Competenze Chiave: Padronanza del modello Input–Elaborazione–Output, gestione rigorosa della memoria RAM (variabili, costanti, sistemi di tipi), algebra booleana, controllo di flusso condizionale (`if/else`), strutture iterative (`while`, `for`), manipolazione di collezioni indicizzate da zero (array e stringhe) e tecniche di debugging sistematico.

2.1 Il Modello Computazionale e la Mappa dei Concetti

All'inizio del secondo anno di laboratorio è essenziale consolidare una visione d'insieme chiara e solida: prima ancora di scegliere una sintassi o un linguaggio specifico (sia esso C++, Python, Java o C#), ogni sistema software poggia su **cinque pilastri logici universali**:

1. **Il Ragionamento (Algoritmi & Logica):** La sequenza ordinata e non ambigua di passi che trasforma i dati in ingresso nella risposta richiesta.
2. **La Memoria (Variabili, Costanti & Tipi):** Le celle fisiche della memoria RAM etichettate da un identificatore, vincolate a uno specifico tipo di dato e deputate a conservare lo stato del sistema.
3. **Le Scelte (Controlli & Bivi Condizionali):** La logica binaria pura (**VERO** / **FALSO**), gli operatori di confronto e le congiunzioni booleane (`AND`, `OR`, `NOT`) che consentono al processore di imboccare percorsi di esecuzione alternativi.
4. **L'Automazione (Cicli & Iterazioni):** La capacità intrinseca dell'elaboratore di ripetere blocchi di istruzioni milioni di volte a velocità strabiliante senza affaticamento, governata da condizioni di permanenza (`while`) o da conteggi definiti (`for`).
5. **Le Collezioni (Vettori & Sequenze):** L'organizzazione di dati contigui in memoria accessibili mediante una posizione numerica (**indice**), rigorosamente calcolata a partire da **zero**.

2.1.1 Che cos'è formalmente un Algoritmo?

Nel linguaggio quotidiano usiamo la nostra intuizione per colmare le lacune delle istruzioni umane. Al contrario, il microcontrollore o il microprocessore del nostro **Vending Machine Manager** non possiede alcuna intuizione, fantasia o capacità di interpretare il non detto: esso esegue passivamente e pedissequamente comandi binari elementari.

Definizione Formale di Algoritmo

Un **algoritmo** è un elenco **finito** e **ordinato** di operazioni **non ambigue** (deterministiche) che, ricevendo un insieme di dati di ingresso (*input*), produce con certezza matematica il risultato voluto (*output*).

2.1.2 Lo Schema Universale: Input → Algoritmo → Output

Qualsiasi sistema informatico, dal più minuscolo firmware per macchinette del caffè fino ai grandi centri dati per il cloud computing, opera secondo lo schema universale di elaborazione. Nel nostro progetto guida annuale la corrispondenza è immediata:

1. INPUT (Dati Esterni) ⟶ [2. ALGORITMO (Elaborazione)] ⟶ 3. OUTPUT (Risultato Tangibile)

- **Input:** Gli stimoli e i dati immessi dall'esterno:
  - Monete inserite nella gettoniera ottico-magnetica (es. `1.50€`).
  - Codice alfanumerico digitato dal cliente sulla tastiera (es. `"B2"`).
  - Segnali booleani provenienti dai sensori fisici (es. finecorsa dello sportello chiuso).
- **Algoritmo (Scatola Nera del Software):** Le regole di business e di controllo interno:
  - Verifica se il codice `"B2"` corrisponde a un prodotto valido nel listino.
  - Controllo se il magazzino interno dispone di scorte residue (`scorte > 0`).
  - Confronto fra credito versato e prezzo nominale (`credito >= PREZZO`).
  - Calcolo del resto dovuto e comando elettrico di attivazione del motore a spirale.
- **Output:** La risposta prodotta verso l'utente e il mondo reale:
  - Caduta meccanica dello snack nel cassetto di ritiro.
  - Erogazione delle monete di resto dalla cassa.
  - Messaggio di riepilogo o cortesia visualizzato sul display LCD.

2.1.3 Le Tre Regole Fondamentali di un Algoritmo

Affinché una procedura logica sia computabile e formalmente corretta, deve soddisfare contemporaneamente tre proprietà inviolabili:

1. **Finitezza:** L'esecuzione deve **terminare obbligatoriamente** dopo un numero finito di passi elementari. Se il motore del distributore girasse all'infinito senza mai arrestarsi, brucerebbe l'avvolgimento del circuito elettrico violando il principio di finitezza.

2. **Definitezza (Non ambiguità):** Ogni singola istruzione deve risultare univoca, priva di dubbi o interpretazioni soggettive. Non possiamo istruire la macchina dicendo *"versa un po' di latte caldo"*: dobbiamo ordinare tassativamente *"apri l'elettrovalvola del latte per 40 millilitri a 65 gradi centigradi"*.
3. **Generalità:** L'algoritmo non deve risolvere un singolo conteggio contingente, bensì deve essere in grado di risolvere **l'intera classe di problemi** per cui è stato concepito.

Il Divieto Assoluto di Hardcoding

L'algoritmo di calcolo del resto del nostro distributore non deve funzionare esclusivamente nel caso in cui un cliente acquisti un caffè da 1.00€ pagando con una moneta da 2.00€. Deve funzionare per **qualsiasi prezzo di listino** e per **qualsiasi combinazione di denaro** inserito!

Scrivere formule con valori fissi cablati a mano nel codice (*hardcoding*) svuota l'informatica della sua utilità:

Formula Universale: `resto = credito_totale - prezzo_prodotto`

Attraverso le **variabili** e i **parametri** il codice diventa universale, resiliente ed eternamente riutilizzabile.

2.2 La Gestione della Memoria: Variabili, Costanti e Tipi

La memoria di lavoro (RAM) può essere paragonata a un immenso casellario postale composto da miliardi di cellette microscopiche. Scrivere codice macchina indicando gli indirizzi esadecimali fisici di queste celle sarebbe insostenibile per la mente umana; per questa ragione i linguaggi di programmazione ad alto livello mettono a disposizione le **variabili**.

La Variabile come Contenitore Tipizzato

Una **variabile** è una porzione di memoria RAM dotata di:

- Un **nome simbolico (identificatore):** scelto dal programmatore secondo nomi autoesplicativi (es. `credito_inserito`).
- Un **tipo di dato:** che determina la dimensione in byte riservata e l'insieme delle operazioni ammissibili.
- Un **valore corrente:** che può essere letto e sovrascritto più volte durante l'esecuzione del programma.

2.2.1 La Dinamica dell'Assegnazione: Da Destra a Sinistra

L'assegnazione è l'azione attiva con cui si deposita un'informazione all'interno di una cella di memoria. In molti linguaggi essa si esprime con il singolo carattere `=` (o in pseudocodice tramite la freccia ←).

La Regola d'Oro dell'Assegnazione

L'istruzione di assegnazione:

1. Valuta e calcola interamente l'espressione situata a **DESTRA** dell'operatore.

2. Deposita (sovrascrivendolo) il valore risultante all'interno della variabile posta a **SINISTRA**.

Si legge e si interpreta sempre rigorosamente da destra verso sinistra!

Evoluzione dello Stato del Credito nel Tempo

Osserviamo come il valore conservato nella variabile `credito` muta dinamicamente mentre l'utente introduce monete successive nel distributore:

```
// 1. All'inizio della transazione la memoria viene azzerata:
credito = 0.00;

// 2. Il cliente inserisce 50 centesimi (0.50 euro):
// Si calcola a destra: 0.00 + 0.50 = 0.50, e si salva a sinistra:
credito = credito + 0.50;  // Ora credito contiene: 0.50

// 3. Il cliente inserisce una moneta da 1.00 euro:
// Si calcola a destra: 0.50 + 1.00 = 1.50, e si salva a sinistra:
credito = credito + 1.00;  // Ora credito contiene: 1.50
```

In matematica l'equazione $x = x + 1$ sarebbe assurda e priva di soluzioni reali; in informatica l'istruzione `x = x + 1` rappresenta invece l'operazione elementare per eccellenza: **l'incremento di stato** della memoria.

2.2.2 Le Costanti: Valori Protetti in Sola Lettura

Non tutti i dati possono o devono essere modificati a piacimento. I prezzi base dei prodotti, i limiti fisici della cassa o i pin hardware di comando del microcontrollore devono rimanere stabili e intoccabili durante l'intera sessione operativa.

Definizione di Costante

Una **costante** (espressa con la parola chiave `const` in C++ o mediante la convenzione di scrittura in **TUTTO MAIUSCOLO**) è una locazione di memoria il cui valore viene fissato in fase di inizializzazione e **non può mai essere sovrascritto** da alcuna istruzione successiva.

```
const float PREZZO_ESPRESSO = 1.00;      // Prezzo immutabile del caffe'
const float PREZZO_SNACK    = 1.50;      // Prezzo immutabile dello snack
const float MAX_CAPIENZA_CASSA = 100.00; // Limite capienza monete
```

L'impiego sistematico delle costanti offre due benefici architetturali immensi:

- **Sicurezza di esecuzione:** Impedisce che una svista di programmazione azzeri per errore il costo di uno snack (rendendo il distributore una fonte di perdite economiche). Se si tenta di scrivere `PREZZO_SNACK = 0.00;`, il compilatore blocca immediatamente il programma segnalando errore.

- **Manutenibilità immediata:** Quando l'inflazione o un cambio fornitore richiede di ritoccare il listino, basta modificare **una sola riga in testa al programma**, anziché ricercare e sostituire a mano il numero `1.50` sparso in centinaia di punti del sorgente.

2.2.3 Il Catalogo dei Tipi di Dato Fondamentali

Il calcolatore non conosce concetti umani come "denaro", "nomi" o "temperature": memorizza solo sequenze di bit (0 e 1). Il **tipo di dato** comunica all'elaboratore quanti byte riservare e come decodificare quei bit.

Tipo di Dato	Nome Tecnico	Natura e Caratteristiche	Esempio nel Vending Machine
Intero	`int`	Valori numerici privi di virgola. Conteggi discreti e indici.	`cialde = 25;` `monete = 48;`
Decimale	`float` / `double`	Numeri reali con virgola mobile. Notazione col **punto decimale** (`.`).	`prezzo = 1.80;` `temperatura = 3.8;`
Booleano	`bool`	Occupa 1 bit logico. Due soli stati: **true** (1) o **false** (0).	`vano_aperto = false;` `snack_caduto = true;`
Testo	`char` / `string`	Singoli caratteri (`'A'`) o sequenze ordinate di simboli (`"B2"`).	`codice = "B2";` `msg = "INSERIRE RESTO";`

Tabella 2.1: Panoramica sinottica dei tipi di dato fondamentali nel software del Vending Machine.

I Booleani e il Mondo Fisico dei Sensori

Nel mondo reale, tutti i sensori elettromeccanici del distributore comunicano con la CPU tramite stati booleani binari:

- Il microswitch dello sportello: `false` se la porta è regolarmente chiusa, `true` se aperta.
- La barriera ottica a infrarossi del vano di caduta: quando lo snack cade, interrompe il fascio luminoso facendo scattare `prodotto_sganciato = true`.

Attenzione: Numero vs Testo (La Trappola del Più)

L'operatore `+` assume comportamenti radicalmente diversi a seconda dei tipi coinvolti:

- **Somma Matematica:** Se operiamo su numeri:

  `10 + 5` $\longrightarrow$ produce il risultato numerico intero **15**.

- **Concatenazione di Testo:** Se operiamo su stringhe racchiuse tra virgolette:

  `"10" + "5"` $\longrightarrow$ produce la parola composta **"105"**!

È il tipo di dato che impone al processore quale micro-operazione hardware eseguire!

2.3 Aritmetica del Calcolatore e l'Operatore Modulo (%)

La gestione contabile di un distributore automatico richiede il calcolo continuo di totali, incassi e differenze:

- **Addizione (+) e Sottrazione (-):** Calcolo immediato dell'avanzo dovuto al cliente:

```
resto = credito_inserito - PREZZO_SNACK;
```

- **Moltiplicazione (*):** Calcolo degli incassi totali prodotti da una corsia:

```
incasso_corsia = lattine_vendute * PREZZO_LATTINA;
```

- **Divisione Reale (/) contro Divisione Intera:** Se dividiamo due numeri decimali otteniamo la media esatta (`totale / 5.0`). Se invece operiamo tra interi (es. in C++), la frazione viene troncata: `7 / 2` produce **3**, non 3.5. Questo è utilissimo per calcolare quanti tubi completi da 10 monete possiamo confezionare: `monete / 10`.

2.3.1 L'Operatore Speciale Modulo (%)

Nel contesto dell'aritmetica discreta e della programmazione, il simbolo % non rappresenta la percentuale, bensì l'operatore matematico di **modulo**.

Definizione di Modulo (%)

L'operatore **modulo (%)** calcola e restituisce il **resto della divisione intera** tra due numeri.

$$\text{Dati due interi } A \text{ e } B: \quad A = (Q \times B) + R \quad \Longrightarrow \quad A \pmod{B} = R$$

L'operatore modulo trova due applicazioni cruciali nel nostro laboratorio:

Applicazione 1: L'Erogazione a Tagli di Moneta

Supponiamo che il distributore debba restituire un resto totale di **70 centesimi** e intenda utilizzare il taglio più grande a disposizione (monete da 50 centesimi):

```
// Quante monete piene da 50 centesimi possiamo restituire?
monete_da_50 = 70 / 50;  // Divisione intera: eroga 1 moneta

// Quanto denaro avanza ancora da restituire con tagli minori?
resto_residuo = 70 % 50; // Operatore modulo: restano 20
    centesimi!
```

Grazie al modulo, il controller sa istantaneamente che dopo la moneta da 50 cent deve attivare il motore delle monete da 20 cent.

Applicazione 2: La Verifica Infallibile di Parità

In qualunque linguaggio, il modulo per 2 rappresenta il test matematico universale per verificare se una cifra è pari o dispari:

```
if (numero % 2 == 0) {
    mostra("Numero PARI (resto della divisione per 2 uguale a
        zero)");
```

```
} else {
    mostra("Numero DISPARI (resto della divisione per 2 uguale
        a uno)");
}
```

2.4 La Logica Booleana e il Controllo di Flusso

I calcolatori elettronici elaborano la realtà mediante l'algebra binaria formulata da George Boole: non esistono gradazioni sfumate come "forse", "probabilmente" o "quasi". Una condizione formulata alla CPU può collassare unicamente in due stati deterministici:

- **VERO (True / 1):** La condizione è pienamente soddisfatta → il blocco di codice subordinato viene mandato in esecuzione.
- **FALSO (False / 0):** La condizione non è rispettata → il blocco di codice subordinato viene completamente ignorato e scavalcato.

2.4.1 Gli Operatori Relazionali di Confronto

Per interrogare lo stato della memoria ed effettuare paragoni si utilizzano gli operatori relazionali:

Operatore	Significato Logico	Esempio nel Vending Machine	Significato nel Dominio
`==`	**Uguale a?**	`credito == 1.50`	Il denaro versato è l'importo esatto
`!=`	**Diverso da?**	`codice_digitato != ""`	Il cliente ha premuto almeno un tast
`>=`	**Maggiore o uguale?**	`credito >= PREZZO`	Il denaro versato è sufficiente all'acq
`>`	**Strettamente maggiore?**	`scorte_prodotto > 0`	Lo snack è fisicamente presente nella
`<=`	**Minore o uguale?**	`monete_cassa <= MAX`	La vaschetta monete non è ancora st
`<`	**Strettamente minore?**	`credito < PREZZO`	Denaro insufficiente: occorre chieder

Tabella 2.2: Operatori relazionali e loro interpretazione nel software gestionale.

2.4.2 Il Tranello Mortale: Assegnazione (=) contro Confronto (==)

Il Bug Più Famoso e Distruttivo della Storia del Software

Uno dei primi e più gravi errori commessi dai programmatori consiste nel confondere l'uguale singolo con il doppio uguale:

- **Singolo Uguale (=) → ASSEGNAZIONE:** È un'istruzione imperativa attiva. Apre la scatola di memoria e sovrascrive il dato:

  `credito = 1.50;` ⟶ *"Scrivi 1.50 all'interno della variabile credito."*

- **Doppio Uguale (==) → CONFRONTO:** È una domanda passiva. Non altera minimamente la memoria:

  `if (credito == 1.50)` ⟶ *"Il credito equivale attualmente a 1.50? (Sì / No)"*

Se in C++ scriviamo erroneamente dentro a un blocco condizionale:
`if (credito = 1.50)` ⟸ **ERRORE GRAVISSIMO!**

Il compilatore esegue l'assegnazione, fissa il credito a 1.50 (cancellando il dato autentico immesso dal cliente) e poi, valutando che 1.50 è diverso da zero, considera la condizione **sempre vera**! Il programma compila senza segnalare errori sintattici ma genera un bug logico silente disastroso.

2.4.3 Gli Operatori Logici Booleani: AND, OR, NOT

Spesso una decisione complessa richiede la combinazione di più premesse contemporanee:

1. L'Operatore Logico AND (&&)

Restituisce **VERO unicamente se TUTTE le condizioni** collegate sono contemporaneamente vere. Se anche una sola condizione risulta falsa, l'intera espressione crolla a **FALSO**:

Credito Sufficiente?	Scorte Residue > 0?	Esito AND (&&)	Comportamento Hardware
VERO	**VERO**	**VERO**	**Eroga snack e sblocca spirale**
VERO	**FALSO**	**FALSO**	Rifiuta: Snack esaurito in vetrina!
FALSO	**VERO**	**FALSO**	Rifiuta: Denaro mancante!
FALSO	**FALSO**	**FALSO**	Rifiuta categorized

```
if (credito >= PREZZO_SNACK && scorte_rimaste > 0) {
    avvia_motore_spirale();
} else {
    mostra("Impossibile procedere: credito scarso o scorte
        esaurite.");
}
```

2. L'Operatore Logico OR (||)

Restituisce **VERO se ALMENO UNA delle condizioni** collegate è verificata. Risulta falso solo nell'unico caso in cui tutti i predicati sono falsi contemporaneamente:

```
// Accettazione delle sole monete di taglio autorizzato:
if (moneta_introdotta == 1.00 || moneta_introdotta == 2.00) {
    credito = credito + moneta_introdotta;
} else {
    espelli_moneta(); // Rifiuta rondelle di ferro o tagli non
        previsti
}
```

3. L'Operatore Logico NOT (!)

È l'operatore unario di **inversione logica**: trasforma il vero in falso e il falso in vero:

```
bool cassa_in_avaria = false;

// Se NON e' in avaria, la macchina opera regolarmente:
if (!cassa_in_avaria) {
    mostra("Distributore Pronto. Inserisci moneta.");
} else {
    mostra("FUORI SERVIZIO PER GUASTO");
}
```

2.4.4 Espressioni Booleane Composte e la Precedenza delle Parentesi

In logica matematica formale, l'operatore **AND ha la precedenza naturale sull'operatore OR** (esattamente allo stesso modo in cui nella normale algebra l'operazione di moltiplicazione ha priorità sull'addizione).

La Regola Aurea delle Parentesi Tonde

Per evitare qualunque ambiguità logica e prevenire errori subdoli, **utilizziamo sempre le parentesi tonde** per raggruppare visivamente e gerarchicamente i criteri decisionali:

```
// L'acquisto e' valido se:
// (L'utente ha pagato con contanti SUFFICIENTI OPPURE con chiavetta valida)
// E CONTEMPORANEAMENTE ci sono scorte nel vano!
if ( (credito >= PREZZO || chiavetta_inserita) && scorte_disponibili > 0 ) {
    eroga_prodotto();
}
```

Senza le parentesi esterne attorno all'OR, la condizione verrebbe letta dal processore come: `credito >= PREZZO || (chiavetta && scorte > 0)`, permettendo l'erogazione in contanti anche se le scorte sono zero!

2.4.5 Le Strutture di Selezione: IF, ELSE e Scelte Multiple

La Selezione Bivalente (IF - ELSE)

Rappresenta il bivio logico fondamentale. Il principio cardine di questa struttura è la **mutua esclusione**: o viene eseguito il ramo del vero, o viene eseguito il ramo del falso. **Non accadrà mai che vengano eseguiti entrambi!**

```
if (credito >= PREZZO_SNACK) {
    mostra("Snack erogato! Ritira il prodotto dal vano.");
    credito = credito - PREZZO_SNACK;
    eroga_resto(credito);
} else {
    float mancanti = PREZZO_SNACK - credito;
    mostra("Credito insufficiente. Inserisci altri centesimi.");
}
```

La Selezione Multipla a Cascata (IF - ELSE IF - ELSE)

Quando le alternative possibili sono più di due (ad esempio la decodifica dei tasti premuti sul tastierino alfanumerico), si organizza una sequenza controllata a cascata:

```
if (codice == "A1") {
    prodotto_selezionato = "Caffe' Espresso";
} else if (codice == "A2") {
    prodotto_selezionato = "Cappuccino";
} else if (codice == "B1") {
    prodotto_selezionato = "Acqua Naturale 500ml";
} else if (codice == "B2") {
    prodotto_selezionato = "Croccantelle al Prosciutto";
} else {
```

```
    // La clausola finale raccoglie qualsiasi combinazione non valida:
    mostra("Errore: codice slot inesistente sul listino!");
}
```

La cascata viene valutata sequenzialmente dall'alto verso il basso. **Nel momento esatto in cui un controllo** senza sprecare cicli di clock a valutare le alternative sottostanti.

2.5 L'Automazione: Cicli e Iterazioni

Scrivere a mano righe di codice identiche ripetute per decine o centinaia di volte è inefficiente, prono a errori di battitura e anti-ingegneristico.

Il Principio DRY (Don't Repeat Yourself)

Uno dei canoni supremi dell'ingegneria del software stabilisce: **non duplicare mai la logica**. Se un'operazione deve ripetersi, si deve ricorrere a una struttura ciclica (*loop*), demandando alla velocità dell'hardware il compito dell'iterazione.

2.5.1 Il Ciclo Pre-Condizionale (WHILE)

Il ciclo **WHILE** (mentre/finché) è la struttura iterativa per eccellenza quando **non si conosce a priori quant** poiché la ripetizione dipende da un evento asincrono esterno (l'utente che infila monete o un sensore che rileva un passaggio).

Funzionamento del Ciclo WHILE

La condizione di test viene esaminata **PRIMA di ogni singolo giro**:

- Se la condizione è **VERA**: esegue tutte le istruzioni del blocco interno e risale in cima a rivalutare la condizione.
- Se la condizione è **FALSA**: il ciclo termina all'istante e l'elaborazione prosegue oltre.
- Se la condizione è falsa sin dal primissimo istante, il blocco interno **non viene eseguito nemmeno una volta**.

```
credito = 0.00;
const float PREZZO_MINIMO = 1.50;

// Continua a chiedere monete finche' la soglia non viene raggiunta:
while (credito < PREZZO_MINIMO) {
    mostra("Credito attuale: " + credito + " euro. Inserisci moneta...");
    credito = credito + leggi_valore_da_gettoniera();
}

mostra("Credito sufficiente! Sblocca la selezione del prodotto.");
```

Allarme Loop Infinito: Il Blocco della Macchina

Se all'interno del corpo del ciclo `WHILE` ci si dimentica di inserire l'istruzione che aggiorna la variabile testata nella condizione:

```
// ESEMPIO DI ERRORE FATALE:
credito = 0.00;
while (credito < 1.50) {
    mostra("Inserisci moneta...");
    // ERRORE: credito non viene MAI incrementato!
}
```

La condizione `0.00 < 1.50` rimarrà vera in eterno. Il programma saturerà il 100% del microprocessore in un vortice senza uscita (**loop infinito**), paralizzando la macchina fino al riavvio forzato.

La Regola di Chiusura: Nel corpo di ogni `while` deve sempre esistere un'istruzione che avvicini la variabile di controllo alla condizione di uscita.

2.5.2 Il Ciclo a Conteggio Prefissato (FOR)

Il ciclo **FOR** è lo strumento naturale da adottare quando il numero di iterazioni da compiere è **noto a priori** (ad esempio far compiere esattamente 3 giri alla spirale meccanica per disincagliare uno snack):

I Tre Elementi Costitutivi del FOR

Ogni ciclo `FOR` governa in un'unica riga compatta tre momenti chiave:

1. **Inizializzazione:** Assegna il valore di partenza al contatore (es. `int giro = 1`). Viene eseguita una sola volta all'avvio.
2. **Condizione di permanenza:** Espressione booleana testata prima di ogni ciclo (es. `giro <= 3`).
3. **Passo di avanzamento:** L'incremento applicato al termine di ciascun giro (es. `giro++`).

```
// Comanda l'avvolgimento del motore a spirale per 3 rotazioni
   complete:
for (int giro = 1; giro <= 3; giro++) {
    attiva_motore_spirale_360_gradi();
    mostra("Completata rotazione numero: " + giro);
}
mostra("Prodotto caduto nel cassetto di ritiro!");
```

2.5.3 Guida di Scelta Architetturale: WHILE o FOR?

2.6 Collezioni e Sequenze: Vettori (Array) e Stringhe

Se dovessimo gestire un distributore con 50 prodotti differenti creando 50 variabili isolate (`p1`, `p2`, ..., `p50`), il nostro codice risulterebbe illeggibile e ingestibile. Per raggruppare dati omogenei la computer science impiega i **vettori (array)**.

Adotta il ciclo WHILE quando...	Adotta il ciclo FOR quando...
Non conosci in anticipo quanti giri occorreranno per terminare il compito.	**Conosci con certezza il numero** prefissato di passaggi da compiere all'avvio.
La fine del ciclo dipende dall'**interazione con l'utente** (es. attendere monete o codici PIN).	Vuoi **scorrere ordinatamente** tutti gli elementi di una collezione, lista o vettore di memoria.
La condizione è guidata da un **sensore fisico** (es. far ruotare la spirale *finché* la fotocellula non rileva l'avvenuta caduta).	Vuoi eseguire un conteggio numerico progressivo o regressivo tra due estremi noti.

Tabella 2.3: Criteri di selezione della struttura iterativa ottimale.

Definizione di Vettore (Array)

Un **vettore (o array)** è una struttura dati lineare composta da una sequenza di celle contigue in memoria RAM:

- Tutte le celle appartengono al **medesimo tipo di dato** (es. tutte stringhe o tutti numeri decimali).
- Ogni elemento è individuato e accessibile mediante la sua posizione numerica, detta **indice**.

2.6.1 La Regola Sacra dell'Indice da ZERO

L'Indice come Offset di Memoria

In informatica, l'indice **NON rappresenta il numero d'ordine umano**, bensì un **offset** (la distanza in celle fisiche a partire dall'indirizzo di base del vettore):

La prima cella del vettore si trova a distanza ZERO dall'inizio: ha indice [0]!

```
// Definizione di un vettore di 3 prodotti:
string vetrina[3] = {"Patatine", "Cioccolato", "Biscotti"};

// Accesso posizionale:
vetrina[0]; // Restituisce "Patatine"   (Offset 0: PRIMO elemento assoluto!)
vetrina[1]; // Restituisce "Cioccolato" (Offset 1: SECONDO elemento)
vetrina[2]; // Restituisce "Biscotti"   (Offset 2: TERZO e ULTIMO elemento)
```

Attenzione: Accesso Fuori Limite (Out of Bounds)

Se un vettore ha dimensione totale pari a N **elementi**, gli unici indici legali e accessibili sono compresi nell'intervallo chiuso:

$$\textbf{Da } 0 \textbf{ fino a } N-1$$

Cercare di accedere all'indice `vetrina[3]` quando il vettore contiene solo 3 elementi costituisce una grave **violazione di memoria** (*Index Out of Bounds* o *Segmentation Fault* in C++). Il programma tenterà di leggere una cella non riservata, causando crash

istantaneo o corruzione imprevedibile dei dati!

2.6.2 Scorrere un Vettore: Il Binomio Perfetto FOR + Array

Il pattern universale per ispezionare, stampare o elaborare tutti gli elementi di una collezione consiste nell'accoppiare un ciclo a conteggio con l'accesso indicizzato:

```
string bevande[3] = {"Espresso", "Ginseng", "Te' al Limone"};
const int N = 3;

// Scansione lineare: l'indice 'i' spazia ordinatamente da 0 a N - 1:
for (int i = 0; i < N; i++) {
    mostra("Corsia " + i + " contiene: " + bevande[i]);
}
```

Cosa compie la CPU passo dopo passo?

- Passo 0: `i = 0` → accede alla cella di base `bevande[0]` ed espone `"Espresso"`.
- Passo 1: `i = 1` → si sposta alla cella `bevande[1]` ed espone `"Ginseng"`.
- Passo 2: `i = 2` → raggiunge la cella finale `bevande[2]` ed espone `"Te' al Limone"`.
- Passo 3: `i = 3` → la condizione `3 < 3` risulta falsa; il ciclo termina in totale sicurezza.

2.6.3 Le Stringhe come Vettori Ordinati di Caratteri

Nel calcolatore anche una stringa di testo (come il codice alfanumerico `"B2"` digitato dal cliente) non è un'entità monolitica, bensì un **vettore ordinato di singoli caratteri** codificati secondo la tabella ASCII:

```
string codice_tasto = "B2";

// Possiamo estrarre i singoli caratteri mediante le parentesi
    quadre:
char lettera_piano   = codice_tasto[0]; // Estrae il carattere 'B'
    (indice 0)
char numero_spirale  = codice_tasto[1]; // Estrae il carattere '2'
    (indice 1)
```

Questo permette al microcontrollore di scomporre la scelta del cliente: il carattere `'B'` indirizza il relè del secondo ripiano, mentre il carattere `'2'` accende l'attuatore della seconda molla rotante.

2.7 Laboratorio Pratico: Le Tre Sfide alla LIM e Debugging

Durante l'attività in classe abbiamo analizzato tre classici errori concettuali che affliggono i software gestionali. Rivediamo le sfide con la loro analisi diagnostica rigorosa:

2.7.1 Sfida 1: Il Tranello nel Controllo Condizionale

Individua il Bug nel Controllo

Analizza il seguente frammento di codice destinato all'accettazione del credito:

```
float moneta = 1.00;
float credito = 0.00;

SE (moneta = 1.00) {
    credito = credito + moneta;
}
```

Quale errore metodologico è stato commesso e quali conseguenze comporta?

Diagnosi e Correzione della Sfida 1

Errore Rilevato: È stato adoperato l'operatore di **assegnazione singola (=)** invece dell'operatore relazionale di **uguaglianza (==)**.

Conseguenza: L'istruzione non interroga il valore di `moneta`, bensì le assegna imperativamente il valore `1.00`. Tale operazione produce come valore risultante `1.00` che, essendo diverso da zero, viene interpretato come **sempre VERO**, inducendo la macchina ad accreditare denaro fittizio anche se nella feritoia fosse stata inserita una moneta contraffatta o di taglio differente.

Istruzione Corretta:

```
SE (moneta == 1.00) {
    credito = credito + moneta;
}
```

2.7.2 Sfida 2: L'Iterazione Paralizzata nel Ciclo

Individua il Bug nel Ciclo

Considera il seguente loop per l'attesa del denaro:

```
float credito = 0.00;
const float PREZZO = 1.50;

MENTRE (credito < PREZZO) {
    mostra("Inserisci moneta da 50 centesimi...");
}
```

Perché questo programma non uscirà mai dall'iterazione?

Diagnosi e Correzione della Sfida 2

Errore Rilevato: Violazione della regola di chiusura del ciclo `WHILE`. All'interno delle parentesi graffe manca totalmente l'istruzione di aggiornamento della variabile di controllo.

Conseguenza: Il valore di `credito` resta inchiodato a `0.00` per sempre. La condizione `0.00 < 1.50` continua a risultare vera all'infinito, bloccando il distributore e intasando il display di messaggi a ciclo continuo (**loop infinito**).

Istruzione Corretta:

```
MENTRE (credito < PREZZO) {
    mostra("Inserisci moneta da 50 centesimi...");
    credito = credito + attendi_e_leggi_moneta(); // Avanzamento di stato!
}
```

2.7.3 Sfida 3: L'Accesso Oltre il Confine del Vettore

Individua il Bug nel Vettore

Osserva il caricamento e l'erogazione dei prodotti:

```
string prodotti[2] = {"Patatine", "Cioccolato"}; // Dimensione: 2 elementi
int scelta = 2;

mostra("Erogazione in corso: " + prodotti[scelta]);
```

Perché questo comando genera un errore fatale a runtime?

Diagnosi e Correzione della Sfida 3

Errore Rilevato: Violazione dei limiti di allocazione del vettore (**Index Out of Bounds**).

Conseguenza: Il vettore contiene $N = 2$ elementi. In virtù della regola sacra dell'indice da zero, le sole posizioni legali sono:

- `prodotti[0]` → corrisponde a `"Patatine"`
- `prodotti[1]` → corrisponde a `"Cioccolato"`

L'indice `[2]` tenta di accedere a una terza cella inesistente nell'array, sconfinando in aree di memoria non autorizzate e provocando il crash immediato dell'applicazione.

Istruzione Corretta: Per selezionare il secondo prodotto (il Cioccolato), l'indice da indicare è **1**:

```
int scelta = 1; // Seleziona il secondo elemento (Cioccolato)
mostra("Erogazione in corso: " + prodotti[scelta]);
```

2.8 Modellazione Completa del Vending Machine Manager

A coronamento della panoramica didattica, integriamo tutti i concetti esaminati (tipi, costanti, assegnazione, logica booleana, selezione e cicli) nella modellazione completa del nostro progetto di laboratorio.

2.8.1 1. Modellazione della Struttura Dati di Memoria

Prima di redigere le istruzioni operative, un buon ingegnere del software organizza la configurazione e le variabili di stato:

```
// ============================================================================
// SEZIONE 1: COSTANTI DI CONFIGURAZIONE (Sola Lettura)
// ============================================================================
const float PREZZO_SNACK       = 1.50;  // Prezzo fisso di vendita
const int   MAX_CAPIENZA_SLOT  = 10;    // Capienza massima spirale
const float VALORE_MINIMO_MONETA = 0.10; // Taglio minimo accettabile

// ============================================================================
// SEZIONE 2: STATO DINAMICO DELLA MACCHINA (Variabili in RAM)
// ============================================================================
float credito_attuale   = 0.00;  // Saldo accumulato per la vendita corrente
int   scorte_rimaste    = 5;     // Scorte residue nello slot selezionato
bool  cassa_in_servizio = true;  // Stato di efficienza del distributore
string codice_selezionato = "B2"; // Codice slot digitato
```

2.8.2 2. L'Algoritmo Unificato di Acquisto ed Erogazione

Ecco l'algoritmo di compravendita completo con controllo di sicurezza a due fattori, aggiornamento del magazzino, calcolo matematico dell'avanzo ed estinzione del credito:

```
// ============================================================================
// SEZIONE 3: ALGORITMO OPERATIVO DI VENDITA
// ============================================================================

// 1. Controllo di fattibilita': la cassa e' attiva, il denaro basta
//    E il cassetto dispone di almeno uno snack disponibile?
if (cassa_in_servizio && credito_attuale >= PREZZO_SNACK &&
    scorte_rimaste > 0) {

    mostra("Transazione autorizzata. Avvio motore a spirale...");

    // 2. Azionamento meccanico: 3 giri per far cadere il pacchetto
    for (int giro = 1; giro <= 3; giro++) {
        ruota_motore_spirale();
    }

    // 3. Scarico dell'inventario di magazzino:
    scorte_rimaste = scorte_rimaste - 1;
    mostra("Snack ritirabile nel cassetto. Scorte rimanenti: " +
        scorte_rimaste);

    // 4. Calcolo rigoroso del resto dovuto:
    float resto_dovuto = credito_attuale - PREZZO_SNACK;

```

```
23      if (resto_dovuto > 0.00) {
24          mostra("Erogazione resto in corso: " + resto_dovuto + "
                euro");
25          eroga_monete_resto(resto_dovuto);
26      }
27
28      // 5. Azzeramento del credito per chiudere la sessione cliente:
29      credito_attuale = 0.00;
30      mostra("Transazione completata con successo. Arrivederci!");
31
32  } else {
33
34      // Gestione mirata dell'anomalia per informare l'utente:
35      if (!cassa_in_servizio) {
36          mostra("Distributore attualmente FUORI SERVIZIO.");
37      } else if (scorte_rimaste <= 0) {
38          mostra("Prodotto esaurito! Premere tasto di reso monete.");
39      } else {
40          float mancano = PREZZO_SNACK - credito_attuale;
41          mostra("Credito insufficiente. Inserire ancora: " + mancano
                + " euro.");
42      }
43  }
```

Questo frammento racchiude l'intera essenza del corso: **variabili** e **costanti** per gestire le risorse, **operatori aritmetici** per la contabilità, **logica booleana composta** per la sicurezza, **cicli definiti** per l'attuazione fisica e **strutture condizionali a cascata** per la comunicazione amichevole con il cliente. Su queste solide basi struttureremo nel prossimo modulo l'architettura a oggetti (**OOP**) del nostro intero gestionale!

Riferimento Rapido e Cheat-Sheet

Tavole sinottiche di riferimento rapido per le convenzioni di codice, comandi Git di frequente utilizzo e pattern di progettazione.